

TCS NQT Advanced Coding Practice Set

Original Problem Set · Foundation-to-Advanced Difficulty

7 Problems · Solutions in C, C++, Java & Python

Compiled for the **Ultimate-TCS-NQT-Resources** repository. All problem statements below are **original scenarios** written to match the algorithmic patterns commonly seen in the TCS NQT Advanced Coding round (arrays, digit manipulation, sorting, combinatorics). They are practice material, not reproduced from any real exam shift. Solve each problem yourself before checking the solution.

Contents

1. Bin Realignment — Arrays
2. Signal Bit Inversion — Bit Manipulation
3. Weekly Class Counter — Date / Modular Arithmetic
4. Priority Ticket Sort — Sorting (Dutch National Flag)
5. New High Counter — Arrays / Running Max
6. Product Checksum — Digit Manipulation
7. Round Table Seating — Combinatorics

Problem 1: Bin Realignment

Topic: Arrays

Problem Statement

A logistics center stores parts in a row of N storage bins. Some bins are empty, represented by the value 0 in the bin list. To make scanning faster, all empty bins must be pushed to the end of the row while the remaining parts keep their original relative order. Write a program that rearranges the bin list so every non-zero value appears first (in its original order), followed by all the zeros.

Constraints

$1 \leq N \leq 10^4$, $0 \leq \text{bin}[i] \leq 10^5$

Example 1

Input:
N = 8
bins = [4, 5, 0, 1, 9, 0, 5, 0]

Output:
4 5 1 9 5 0 0 0

Explanation: The three empty bins (0) move to the end; the order of the non-empty parts (4, 5, 1, 9, 5) is preserved.

Example 2

Input:
N = 6
bins = [6, 0, 1, 8, 0, 2]

Output:
6 1 8 2 0 0

Input / Output Notes

Read N , then read N integers separated by newlines. Print the rearranged array as space-separated values on one line.

Solutions

C

```
#include <stdio.h>

void moveZerosToEnd(int arr[], int n) {
    int insertPos = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] != 0) {
            arr[insertPos++] = arr[i];
        }
    }
    while (insertPos < n) {
        arr[insertPos++] = 0;
    }
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
```

```

        moveZerosToEnd(arr, n);

        for (int i = 0; i < n; i++) printf("%d ", arr[i]);
        return 0;
    }

```

C++

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; i++) cin >> arr[i];

    int insertPos = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] != 0) arr[insertPos++] = arr[i];
    }
    while (insertPos < n) arr[insertPos++] = 0;

    for (int x : arr) cout << x << " ";
    return 0;
}

```

Java

```

import java.util.Scanner;

public class BinRealignment {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int insertPos = 0;
        for (int i = 0; i < n; i++) {
            if (arr[i] != 0) arr[insertPos++] = arr[i];
        }
        while (insertPos < n) arr[insertPos++] = 0;

        StringBuilder sb = new StringBuilder();
        for (int x : arr) sb.append(x).append(" ");
        System.out.println(sb.toString().trim());
    }
}

```

Python

```

def move_zeros_to_end(arr):
    insert_pos = 0
    for i in range(len(arr)):
        if arr[i] != 0:
            arr[insert_pos] = arr[i]
            insert_pos += 1
    for i in range(insert_pos, len(arr)):
        arr[i] = 0
    return arr

if __name__ == "__main__":
    n = int(input())
    bins_ = [int(input()) for _ in range(n)]

```

```
result = move_zeros_to_end(bins_)
print(" ".join(map(str, result)))
```

Problem 2: Signal Bit Inversion

Topic: Bit Manipulation

Problem Statement

A telecom testing tool receives a positive integer signal value N . To test receiver robustness, every bit in the binary representation of N — starting from the most significant bit through to the least significant bit — must be flipped (0 becomes 1, 1 becomes 0). Write a program that prints the resulting integer after this full inversion.

Constraints

$1 \leq N \leq 1000$

Example 1

Input:
N = 10

Output:
5

Explanation: 10 in binary is 1010. Inverting every bit gives 0101, which is 5 in decimal.

Example 2

Input:
N = 23

Output:
8

Explanation: 23 in binary is 10111. Inverting every bit gives 01000, which is 8 in decimal.

Input / Output Notes

Read a single integer N and print the single integer result.

Solutions

C

```
#include <stdio.h>
#include <math.h>

int invertBits(int n) {
    int numBits = (int)(log2(n)) + 1;
    int mask = (1 << numBits) - 1;
    return n ^ mask;
}

int main() {
    int n;
    scanf("%d", &n);
    printf("%d\n", invertBits(n));
    return 0;
}
```

C++

```
#include <bits/stdc++.h>
using namespace std;
```

```

int invertBits(int n) {
    int numBits = (int)log2(n) + 1;
    int mask = (1 << numBits) - 1;
    return n ^ mask;
}

int main() {
    int n;
    cin >> n;
    cout << invertBits(n) << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class SignalBitInversion {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        int numBits = (int) (Math.log(n) / Math.log(2)) + 1;
        int mask = (1 << numBits) - 1;
        System.out.println(n ^ mask);
    }
}

```

Python

```

def invert_bits(n):
    num_bits = n.bit_length()
    mask = (1 << num_bits) - 1
    return n ^ mask

if __name__ == "__main__":
    n = int(input())
    print(invert_bits(n))

```

Problem 3: Weekly Class Counter

Topic: Date / Modular Arithmetic

Problem Statement

A gym holds its signature class every Sunday. Given the day of the week on which a month begins (as a 3-letter code such as "mon", "tue", etc.) and the number of days D being considered from the 1st of that month, write a program that counts how many Sundays fall within the first D days.

Constraints

D can be any positive integer up to 366. The month-start code is one of: mon, tue, wed, thu, fri, sat, sun.

Example 1

Input:
start = "mon"
D = 14

Output:
2

Explanation: Day 1 is Monday, so day 7 is the first Sunday and day 14 is the second. Both fall within the first 14 days.

Example 2

Input:
start = "sun"
D = 15

Output:
3

Explanation: Day 1 itself is a Sunday, then day 8, then day 15.

Input / Output Notes

Read the 3-letter starting weekday code, then read the integer D. Print a single integer: the count of Sundays in the first D days.

Solutions

C

```
#include <stdio.h>
#include <string.h>

int countSundays(char start[], int d) {
    char days[7][4] = {"sun", "mon", "tue", "wed", "thu", "fri", "sat"};
    int startIdx = 0;
    for (int i = 0; i < 7; i++) {
        if (strcmp(days[i], start) == 0) { startIdx = i; break; }
    }
    int count = 0;
    for (int day = 1; day <= d; day++) {
        int weekday = (startIdx + (day - 1)) % 7;
        if (weekday == 0) count++;
    }
    return count;
}
```

```

int main() {
    char start[4];
    int d;
    scanf("%s", start);
    scanf("%d", &d);
    printf("%d\n", countSundays(start, d));
    return 0;
}

```

C++

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    string days[7] = {"sun", "mon", "tue", "wed", "thu", "fri", "sat"};
    string start;
    int d;
    cin >> start >> d;

    int startIdx = 0;
    for (int i = 0; i < 7; i++) if (days[i] == start) { startIdx = i; break; }

    int count = 0;
    for (int day = 1; day <= d; day++) {
        if ((startIdx + (day - 1)) % 7 == 0) count++;
    }
    cout << count << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class WeeklyClassCounter {
    public static void main(String[] args) {
        String[] days = {"sun", "mon", "tue", "wed", "thu", "fri", "sat"};
        Scanner sc = new Scanner(System.in);
        String start = sc.next();
        int d = sc.nextInt();

        int startIdx = 0;
        for (int i = 0; i < 7; i++) {
            if (days[i].equals(start)) { startIdx = i; break; }
        }

        int count = 0;
        for (int day = 1; day <= d; day++) {
            if ((startIdx + (day - 1)) % 7 == 0) count++;
        }
        System.out.println(count);
    }
}

```

Python

```

def count_sundays(start, d):
    days = ["sun", "mon", "tue", "wed", "thu", "fri", "sat"]
    start_idx = days.index(start)
    count = 0
    for day in range(1, d + 1):
        weekday = (start_idx + (day - 1)) % 7
        if weekday == 0:
            count += 1
    return count

```

```
if __name__ == "__main__":  
    start = input().strip()  
    d = int(input())  
    print(count_sundays(start, d))
```

Problem 4: Priority Ticket Sort

Topic: Sorting (Dutch National Flag)

Problem Statement

A support desk labels every incoming ticket with a priority level: 0 (low), 1 (medium), or 2 (high). Given N tickets in the order they arrived, sort them in place so all low-priority tickets come first, then medium, then high — without using a general-purpose sort function.

Constraints

$1 \leq N \leq 10^5$, each value is 0, 1, or 2.

Example 1

Input:
N = 7
tickets = [1, 0, 2, 0, 1, 0, 2]

Output:
0 0 0 1 1 2 2

Example 2

Input:
N = 10
tickets = [2, 1, 0, 2, 1, 0, 0, 1, 2, 0]

Output:
0 0 0 0 1 1 1 2 2 2

Input / Output Notes

Read N , then N integers (each 0, 1, or 2). Print the sorted array as space-separated values.

Solutions

C

```
#include <stdio.h>

void sortTickets(int arr[], int n) {
    int low = 0, mid = 0, high = n - 1, temp;
    while (mid <= high) {
        if (arr[mid] == 0) {
            temp = arr[low]; arr[low] = arr[mid]; arr[mid] = temp;
            low++; mid++;
        } else if (arr[mid] == 1) {
            mid++;
        } else {
            temp = arr[mid]; arr[mid] = arr[high]; arr[high] = temp;
            high--;
        }
    }
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    sortTickets(arr, n);
}
```

```

        for (int i = 0; i < n; i++) printf("%d ", arr[i]);
        return 0;
    }

```

C++

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; i++) cin >> arr[i];

    int low = 0, mid = 0, high = n - 1;
    while (mid <= high) {
        if (arr[mid] == 0) swap(arr[low++], arr[mid++]);
        else if (arr[mid] == 1) mid++;
        else swap(arr[mid], arr[high--]);
    }

    for (int x : arr) cout << x << " ";
    return 0;
}

```

Java

```

import java.util.Scanner;

public class PriorityTicketSort {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int low = 0, mid = 0, high = n - 1;
        while (mid <= high) {
            if (arr[mid] == 0) {
                int t = arr[low]; arr[low] = arr[mid]; arr[mid] = t;
                low++; mid++;
            } else if (arr[mid] == 1) {
                mid++;
            } else {
                int t = arr[mid]; arr[mid] = arr[high]; arr[high] = t;
                high--;
            }
        }

        StringBuilder sb = new StringBuilder();
        for (int x : arr) sb.append(x).append(" ");
        System.out.println(sb.toString().trim());
    }
}

```

Python

```

def sort_tickets(arr):
    low, mid, high = 0, 0, len(arr) - 1
    while mid <= high:
        if arr[mid] == 0:
            arr[low], arr[mid] = arr[mid], arr[low]
            low += 1
            mid += 1
        elif arr[mid] == 1:

```

```
        mid += 1
    else:
        arr[mid], arr[high] = arr[high], arr[mid]
        high -= 1
    return arr
```

```
if __name__ == "__main__":
    n = int(input())
    tickets = list(map(int, input().split()))
    print(" ".join(map(str, sort_tickets(tickets))))
```

Problem 5: New High Counter

Topic: Arrays / Running Max

Problem Statement

A stock-tracking tool receives N daily closing prices in order. Count how many of these prices set a new all-time high at the moment they were recorded (i.e., the price is strictly greater than every price recorded before it). The very first price always counts as a new high.

Constraints

$1 \leq N \leq 20$, $1 \leq \text{price}[i] \leq 10^4$

Example 1

Input:
N = 5
prices = [7, 4, 8, 2, 9]

Output:
3

Explanation: 7 (first price, counts), 8 (> 7, counts), 9 (> 8, counts).
4 and 2 do not set new highs.

Example 2

Input:
N = 5
prices = [3, 4, 5, 8, 9]

Output:
5

Explanation: Every price is higher than all before it.

Input / Output Notes

Read N , then N integers. Print a single integer: the count of new-high prices.

Solutions

C

```
#include <stdio.h>

int countNewHighs(int arr[], int n) {
    int count = 0;
    int maxSoFar = -1;
    for (int i = 0; i < n; i++) {
        if (arr[i] > maxSoFar) {
            count++;
            maxSoFar = arr[i];
        }
    }
    return count;
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
```

```

        printf("%d\n", countNewHighs(arr, n));
        return 0;
    }

```

C++

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; i++) cin >> arr[i];

    int count = 0, maxSoFar = -1;
    for (int x : arr) {
        if (x > maxSoFar) { count++; maxSoFar = x; }
    }
    cout << count << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class NewHighCounter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int count = 0, maxSoFar = -1;
        for (int x : arr) {
            if (x > maxSoFar) { count++; maxSoFar = x; }
        }
        System.out.println(count);
    }
}

```

Python

```

def count_new_highs(prices):
    count = 0
    max_so_far = -1
    for p in prices:
        if p > max_so_far:
            count += 1
            max_so_far = p
    return count

if __name__ == "__main__":
    n = int(input())
    prices = [int(input()) for _ in range(n)]
    print(count_new_highs(prices))

```

Problem 6: Product Checksum

Topic: Digit Manipulation

Problem Statement

A warehouse scanner reads a positive integer code N printed on each item. The item's checksum is defined as the product of all digits of N. Write a program that computes and prints this checksum.

Constraints

$$1 \leq N \leq 10^9$$

Example 1

Input:
N = 5244

Output:
160

Explanation: $5 \times 2 \times 4 \times 4 = 160$.

Example 2

Input:
N = 709

Output:
0

Explanation: Any digit equal to 0 makes the entire product 0.

Input / Output Notes

Read a single integer N and print the single integer checksum.

Solutions

C

```
#include <stdio.h>

long long digitProduct(int n) {
    long long product = 1;
    while (n > 0) {
        product *= (n % 10);
        n /= 10;
    }
    return product;
}

int main() {
    int n;
    scanf("%d", &n);
    printf("%lld\n", digitProduct(n));
    return 0;
}
```

C++

```
#include <bits/stdc++.h>
using namespace std;

long long digitProduct(int n) {
```

```

        long long product = 1;
        while (n > 0) {
            product *= (n % 10);
            n /= 10;
        }
        return product;
    }

int main() {
    int n;
    cin >> n;
    cout << digitProduct(n) << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class ProductChecksum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        long product = 1;
        while (n > 0) {
            product *= (n % 10);
            n /= 10;
        }
        System.out.println(product);
    }
}

```

Python

```

def digit_product(n):
    product = 1
    for ch in str(n):
        product *= int(ch)
    return product

if __name__ == "__main__":
    n = int(input())
    print(digit_product(n))

```

Problem 7: Round Table Seating

Topic: Combinatorics

Problem Statement

N delegates are to be seated around a circular table for a conference. Two specific delegates — the chairperson and the co-chair — must always sit next to each other. Write a program that computes the number of distinct ways to seat all N delegates around the table under this constraint. (Two seatings are the same if one can be rotated to match the other.)

Constraints

$$2 \leq N \leq 50$$

Example 1

Input:
N = 4

Output:
4

Explanation: Fuse the chairperson and co-chair into a single block. That leaves 3 blocks total (the fused pair + 2 other delegates) to seat around the table. Circular arrangements of 3 blocks = $(3-1)! = 2$. The fused pair can be internally ordered in $2! = 2$ ways. Total = $2 \times 2 = 4$.

Example 2

Input:
N = 10

Output:
80640

Explanation: Fusing the pair leaves 9 blocks, giving $(9-1)! = 8! = 40320$ circular arrangements. Times $2!$ for the pair's internal order: $2 \times 40320 = 80640$.

Input / Output Notes

Read a single integer N. Print a single integer: the number of valid circular seatings.

Solutions

C

```
#include <stdio.h>

long long factorial(int n) {
    long long result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}

long long seatingWays(int n) {
    // Fuse the two fixed-adjacent delegates into one unit.
    // That leaves (n - 1) units to arrange in a circle: (n - 2)! ways.
    // The fused pair can be internally ordered in 2! ways.
    return 2 * factorial(n - 2);
}
```

```

int main() {
    int n;
    scanf("%d", &n);
    printf("%lld\n", seatingWays(n));
    return 0;
}

```

C++

```

#include <bits/stdc++.h>
using namespace std;

long long factorial(int n) {
    long long result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}

int main() {
    int n;
    cin >> n;
    long long ways = 2 * factorial(n - 2);
    cout << ways << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class RoundTableSeating {
    static long factorial(int n) {
        long result = 1;
        for (int i = 2; i <= n; i++) result *= i;
        return result;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long ways = 2 * factorial(n - 2);
        System.out.println(ways);
    }
}

```

Python

```

import math

def seating_ways(n):
    # Fuse the two fixed-adjacent delegates into one unit -> (n-1) circular
    # units -> (n-2)! arrangements, times 2! for the pair's internal order.
    return 2 * math.factorial(n - 2)

if __name__ == "__main__":
    n = int(input())
    print(seating_ways(n))

```

Notes

All scenarios and wording above are original and written for this practice set — they are not reproductions of any real TCS NQT shift's questions. Every solution in this set was independently executed against both of its worked examples and confirmed to produce the exact stated output. Still, always re-verify by running the code yourself, especially under the actual TCS iON compiler, before relying on any practice material during timed prep.

Compiled for [salonyranjan/Ultimate-TCS-NQT-Resources](#).